

Communicator 1

Project structure and how the code works

JavaFX desktop chat · SQLite users/conversations · Discord as encrypted message relay

Document generated from the communicator_1 codebase · 7 September 2026

1. What this application is

Communicator 1 is a JavaFX desktop messenger. Users log in against a shared SQLite database. When two users open a chat, the app looks up (or creates) a 1:1 **conversation** with a UUID and an AES encryption key. Live messages are **not stored in SQLite**. They are encrypted, posted to one Discord text channel, and delivered back into every open chat window.

The program starts two login windows on purpose (`HelloApplication`) so two accounts can be tested on the same machine in one process.

2. Layered structure

The project follows a simple layered layout. UI never talks to JDBC directly except through repositories.

Layer	Package	Responsibility
Entry	<code>dk.easv.communicator_1</code>	<code>Launcher</code> starts JavaFX. <code>HelloApplication</code> opens two login stages.
UI	<code>...gui</code> and <code>FXML</code>	<code>LoginController</code> / <code>HelloController</code> bind screens and user actions.
UI helper	<code>...UIHelper</code>	Loads the chat <code>FXML</code> , passes the logged-in user, starts the Discord bot listener.
Services	<code>...Service</code>	Login, conversations, encryption, Discord send/receive.
Business entities	<code>...be</code>	Plain objects: <code>user</code> , <code>conversations</code> , <code>conversation_members</code> , <code>messages</code> .
Data access	<code>repo.repositories</code> / <code>repo.impl</code>	Interfaces plus SQLite implementations.
Connection	<code>...ConnectionManager</code>	Opens JDBC connections to the SQLite file.

Important classes

Class	Role
Launcher	main method. Calls <code>Application.launch(HelloApplication.class)</code> .
HelloApplication	Opens two independent login windows (<code>Login1</code> / <code>Login2</code> titles are unused; both load <code>login-panel.fxml</code>).
LoginController	Login / register form. On success, <code>UiHelper.openChatPanel</code> and closes the login stage.
HelloController	Chat screen: user list, open conversation, send, receive, history.
LoginService + UserRepo	Validate fields, BCrypt password check, load users, assign <code>discord_uuid</code> if missing.
ChatService + ConversationRepo	Find or create the 1:1 conversation; look up by UUID; membership checks.
DiscordBot	One shared JDA client. Registers every <code>HelloController</code> and dispatches ciphertext to them.
MessageService	Encrypt/send, parse Discord payloads, load last 100 channel messages as history, skip local echoes.
MessageCrypto	AES-256-GCM. Random 32-byte key, 12-byte IV packed in front of ciphertext, URL-safe Base64.

3. Screens (FXML)

`login-panel.fxml` is bound to `LoginController`: username, password, optional email, login/register toggle.

`chat-panel.fxml` is a `BorderPane` bound to `HelloController`:

- Left: `userLister` — other users.
- Center: logged-in name, read-only transcript `txtChatId`, input `txtMessagePanelId`, Send button calling `btnSend`.

4. Database

JDBC URL is relative: `jdbc:sqlite:../../DataGripProjects/default/identifier.sqlite`. From the project folder that resolves to the DataGrip SQLite file under the user home. Connections enable WAL and a busy timeout so two chat windows can share the file.

Table	What it stores	Used by live chat?
users	id , username, bcrypt hash, email, admin flag, discord_uuid	Yes — identity and sender UUID
conversations	conversation_id , conversation_uuid , encryption_key , created_at	Yes — routing + crypto
conversation_members	which users belong to which conversation	Yes — receive filtering
conversation_pair	unique pair (user_low, user_high) → conversation_id	Yes — both sides must share one UUID/key
messages	older local rows (id, conversation_id, sender_id, content, sent_at)	No — current send path does not insert here

SQLite vs MySQL syntax. `conversations` was originally created as `INT AUTO_INCREMENT PRIMARY KEY`. SQLite does not treat that as a rowid alias, so `conversation_id` stayed NULL. Members/pair rows used the internal rowid, joins failed, and each window created a new UUID/key. Receive then dropped the message. `ConversationRepo` now repairs that schema to `INTEGER PRIMARY KEY AUTOINCREMENT` and inserts with `RETURNING conversation_id`.

5. Startup and login, step by step

```

Launcher.main
  → HelloApplication.start
    → openWindow() twice (two login stages)

User clicks Login
  → LoginController.LoginRegisterOnClick
    → LoginService.login(username, password)
      → UserRepo.loginUser
        SELECT user by username
        BCrypt.checkpw(typed password, password_hash)
        if discord_uuid empty: generate UUID and UPDATE users
    → UiHelper.openChatPanel(loggedInUser)
      load chat-panel.fxml
      HelloController.setCurrentUser(...)
      DiscordBot.start(controller)
      show chat stage
    → close login stage

```

Register hashes the password with BCrypt (cost 12) and calls `UserRepo.createUser`, also storing a new `discord_uuid`. That UUID is what later appears as the sender id inside encrypted payloads — not the Discord account id, despite the column name.

`DiscordBot.start` appends the chat controller to a static list. The JDA bot is created only once per process. Both windows therefore share one Discord connection and one listener (`MessageService`).

6. Opening a conversation

After login, `HelloController.loadUsers` loads everyone except the current user. Selecting a row runs `openConversation` on a background JavaFX `Task`.

```
ChatService.openConversation(currentId, otherId)
→ ConversationRepo.findOrCreateConversation
    BEGIN IMMEDIATE (SQLite write lock, avoids two inserts for the same pair)
    userLow = min(id1, id2)
    userHigh = max(id1, id2)

    1) conversation_pair lookup for (userLow, userHigh)
    2) else conversation_members join (both user ids on same conversation)
       then INSERT OR IGNORE into conversation_pair
    3) else create:
       new conversation_uuid (random UUID)
       new encryption_key (32 random bytes, Base64)
       INSERT conversations RETURNING conversation_id
       insert both members
       insert pair row
    ensure uuid + key are non-empty
    COMMIT

HelloController stores activeConversation
→ MessageService.loadConversationHistory(uuid, key)
    fetch last 100 Discord messages in the relay channel
    keep only payloads whose header UUID matches this conversation
    decrypt with this conversation key
    append "You:" / "OtherUser:" lines to the transcript
```

The pair table is the source of truth so Alice→Bob and Bob→Alice resolve to the same `conversation_id`, UUID, and key. That is required for decrypt to succeed.

7. Sending a message

`btnSend` refuses empty text and refuses send if no user/conversation is selected. Then:

```
plaintext = senderUuid + "\n" + typed text
ciphertext = MessageCrypto.encrypt(conversation.encryption_key, plaintext)
              AES-256-GCM, random 12-byte IV prepended, URL-safe Base64

locallyDispatched.add(conversationUuid + "|" + ciphertext)
DiscordBot.dispatch(uuid, ciphertext) // in-process, every open HelloController
Discord channel.sendMessage("[APP|c=" + uuid + "]" + ciphertext)

On send success (JavaFX thread): append "You: ..." locally
```

Wire format on Discord looks like:

```
[APP|c=bfb3cc85-b250-42e4-83c7-6cb56b721792] <base64 iv+ciphertext>
```

The regex in `MessageService` is `[APP|c=([^\]]+)\s*(.*)` (dotall). Group 1 is the conversation UUID. Group 2 is the ciphertext.

In-process `dispatch` exists because two windows share one bot. JDA often does not fire `onMessageReceived` for the bot's own message, so the other window would never hear the send if it only waited on Discord.

8. Receiving a message

8.1 From Discord

```
MessageService.onMessageReceived
    ignore if not the configured relay channel
    ignore if body is not an [APP|c=...] payload
    echoKey = uuid + "|" + ciphertext
    if echoKey was already sent by this process:
        remove it and stop          // avoid showing the Discord echo twice
    else:
        DiscordBot.dispatch(uuid, ciphertext)
```

8.2 Inside each chat window

```
HelloController.onDiscordMessage(uuid, ciphertext)
    need currentUser and selectedUser (a chat must be open)

    resolve conversation:
        if activeConversation.uuid matches incoming uuid → use it
        else ChatService.findByUuid(uuid)

    drop unless:
        this window is already viewing that uuid
        OR both currentUser and selectedUser are members of that conversation

    decrypt with that conversation's encryption_key
    drop if decrypt fails (wrong key / truncated payload)
    drop if senderUuid == currentUser.uuid    (own message; UI already showed "You:")

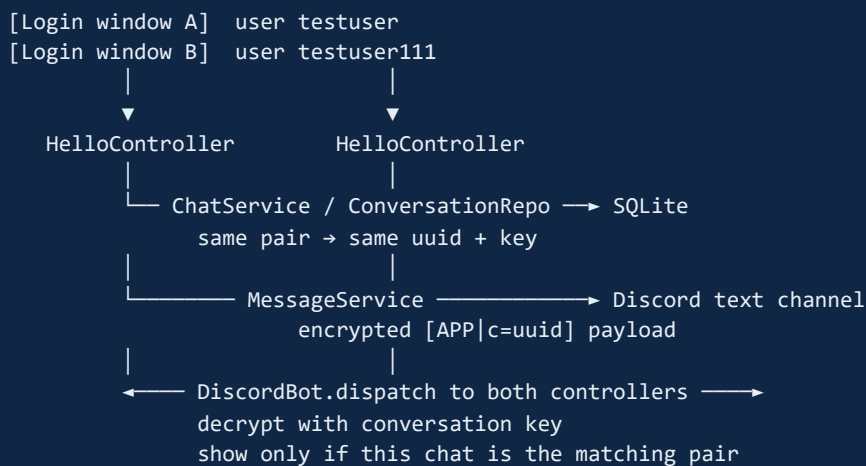
    set activeConversation to the resolved row (keeps both windows on the same uuid)
    append "OtherUser: text" on the JavaFX thread
```

Membership checks use `conversation_members`. That lets a window accept a payload even if it had briefly opened a duplicate conversation row, as long as both people in the open chat belong to the incoming conversation.

9. Encryption details

- Algorithm: AES/GCM/NoPadding, 128-bit authentication tag.
- Key: 32 random bytes, stored on the conversation row (shared secret for that pair).
- IV: 12 random bytes per message, not reused; stored with the ciphertext.
- Plaintext layout: `senderUuid`, newline, message body. That is how the receiver knows who sent it without putting usernames in the Discord header.
- Failed decrypt returns `null` and the UI stays silent (typical when UUID/key do not match).

10. End-to-end picture



11. Background threads

Database and Discord I/O run in daemon threads via JavaFX `Task`. UI updates use `Platform.runLater` or `task.setOnSucceeded`. That keeps the chat window responsive while SQLite and JDA work.

12. What is intentionally not used (yet)

- `be.messages` and the `messages` table — leftover from an earlier DB-only design. Live chat uses Discord history instead.
- `Admin` / `Roles` — user objects expose a role, but the chat UI does not branch on it.
- Direct peer-to-peer sockets — all transport is the one Discord channel, distinguished by conversation UUID.

13. How to run a two-user test

1. Start the app (two login windows appear).
2. Log in as two different users.
3. In each window, select the other user so both open the same pair conversation.
4. Type and send. The sender sees `You:` immediately. The other window should show `Name:` after in-process dispatch (and Discord is used for history / a second process).

This PDF describes the code as of 7 September 2026. It does not include secrets (bot tokens, passwords, or encryption keys).